Reviewer Pack — Sandpile 2-Rank Lower-Bounds Tree-Resolution Size of Tseitin

Entry 82f0b7df0af9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 19:28:33 UTC

Sandpile 2-Rank Lower-Bounds Tree-Resolution Size of Tseitin

Entry ID: `82f0b7df0af9` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-07 19:28:33 UTC

1. Conjecture

Field A (mathematical branch): Chip-firing / Sandpile (critical) groups of graphs **Field B** (complexity object): Tree-resolution refutation size (sub-Frege; gateway to Frege depth lower bounds)

Statement:

> Let G=(V,E) be a connected graph with odd-parity charge c:V→F_2, and let T(G,c) be the (unsatisfiable) Tseitin CNF. Let K(G)=$Z^{\{|V|-1\}}/\tilde{L}(G)Z^{\{|V|-1\}}$ denote the sandpile group of G, computed from the reduced Laplacian L̃(G) (delete one row/column of the combinatorial Laplacian) via Smith Normal Form yielding invariant factors s_1|s_2|...|s_{|V|-1}; define the dyadic rank r_2(G)=#{i: s_i is even}. Then the minimum tree-resolution refutation size satisfies size_TR(T(G,c)) ≥ |V|·2^{r_2(G)}, and a single graph G with size_TR(T(G,c)) < |V|·2^{r_2(G)} refutes the conjecture.

Rationale (proposer's reasoning):

> The sandpile group encodes global cycle/flow structure of G as integer torsion, and its 2-Sylow part precisely measures independent F_2-cycle classes — the same algebraic obstruction that makes Tseitin unsatisfiable. Heuristically each dyadic invariant factor furnishes an independent parity certificate that a tree refutation must dispatch via an additional case-split, doubling the tree size. Chip-firing has been linked to spanning-tree counts and graph Riemann-Roch but, to our knowledge, never to Tseitin proof size, providing a virgin algebraic-combinatorial bridge to Frege-style lower bounds.

Taxonomy category: `PROOF_COMPLEXITY_EXT_FREGE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `981cf7fa3c81949b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 30 seeds × 8 sizes × 3 densities (720 instances), conjecture is SUPPORTED iff every instance satisfies size_TR ≥ |V|·2^{r_2(G)} (support_fraction = 1.0) AND mean Spearman ρ(r_2, log2 size_TR | |V|) ≥ 0.3; FALSIFIED if any single instance violates the bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Tseitin formulas tree resolution sandpile group` - `Tseitin tree-like resolution lower bounds Laplacian Smith normal form` - `chip-firing critical group proof complexity Tseitin 2-rank`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=2.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_graph(n, density):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < density:
                    edges.add((i, j))
        return list(edges)

    def is_connected(graph, n):
        visited = [False] * n
        stack = [0]
        while stack:
            node = stack.pop()
            if not visited[node]:
                visited[node] = True
                for neighbor in graph:
                    if neighbor[0] == node and not visited[neighbor[1]]:
                        stack.append(neighbor[1])
                    elif neighbor[1] == node and not visited[neighbor[0]]:
                        stack.append(neighbor[0])
        return all(visited)

    def add_odd_parity_charge(graph, n):
        charge = [random.choice([0, 1]) for _ in range(n)]
```

```
    for i in range(n):
        parity = sum(charge[j] for j, (_, j) in enumerate(graph) if j != i) % 2
        charge[i] += parity
    return charge

def tseitin_cnf(graph, n, charge):
    cnf = []
    literals = {i: [0, 1] for i in range(n)}
    for i in range(n):
        clause = [literals[i][charge[i]]]
        for j, (_, j) in enumerate(graph):
            if j == i:
                continue
            clause.append(literals[j][1 - charge[j]])
        cnf.append(clause)
    return cnf

def dpll(cnf, literals):
    def propagate():
        while True:
            changed = False
            for clause in cnf:
                if len(clause) == 0:
                    return None
                unit_clauses = [l for l in clause if literals[l] == -1]
                if unit_clauses:
                    literal = unit_clauses[0]
                    literals[literal] = 1 - (literal % 2)
                    changed = True
            if not changed:
                break

    def backtrack(level):
        while level < len(cnf) and literals[cnf[level][0]] != -1:
            level += 1
        if level == len(cnf):
            return True
        literal = cnf[level][0]
        literals[literal] = 1 - (literal % 2)
        if backtrack(level + 1):
            return True
        literals[literal] = -1
        literals[1 - literal] = -1
        return backtrack(level + 1)

    propagate()
    if None in cnf:
        return None
    if backtrack(0):
        return literals

def smith_normal_form(matrix):
    n = len(matrix)
    for i in range(n):
        pivot_row = i
        while matrix[pivot_row][i] == 0:
```

```
            pivot_row += 1
            if pivot_row == n:
                return [0] * n
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
        for j in range(n):
            if j != i:
                factor = matrix[j][i] // matrix[i][i]
                for k in range(i, n):
                    matrix[j][k] -= factor * matrix[i][k]
    return [matrix[i][i] for i in range(n)]

def laplacian_matrix(graph, n):
    laplacian = [[0] * n for _ in range(n)]
    degree = [sum(1 for _, j in graph if j == i) for i in range(n)]
    for i, (_, j) in enumerate(graph):
        laplacian[i][j] = -1
        laplacian[j][i] = -1
    for i in range(n):
        laplacian[i][i] = degree[i]
    return laplacian

def reduced_laplacian(laplacian, n):
    return [row[1:] for row in laplacian[1:]]

def tree_resolution_size(cnf):
    literals = {i: -1 for i in range(2 * len(cnf))}
    stack = []
    while cnf:
        clause = cnf.pop()
        if len(clause) == 0:
            return None
        unit_clauses = [l for l in clause if literals[l] == -1]
        if unit_clauses:
            literal = unit_clauses[0]
            literals[literal] = 1 - (literal % 2)
            stack.append(literal)
        else:
            literal = min(clause, key=lambda x: literals[x])
            literals[literal] = 1 - (literal % 2)
            for j in range(len(cnf)):
                if literal in cnf[j]:
                    cnf[j].remove(literal)
    return len(stack)

n_values = [6, 8, 10, 12, 14, 16, 18, 20]
densities = [3 + i for i in range(3)]
results = []

for n in n_values:
    for density in densities:
        graph = generate_graph(n, density)
        if not is_connected(graph, n):
            continue
        charge = add_odd_parity_charge(graph, n)
        cnf = tseitin_cnf(graph, n, charge)
        literals = dpll(cnf, {})
```

```
            size_TR = tree_resolution_size(cnf) if literals else None
            if size_TR is not None:
                laplacian = laplacian_matrix(graph, n)
                reduced_lap = reduced_laplacian(laplacian, n)
                snf = smith_normal_form(reduced_lap)
                r_2 = sum(1 for s in snf if s % 2 == 0)
                results.append({
                    "n": n,
                    "density": density,
                    "size_TR": size_TR,
                    "r_2": r_2
                })

    mean_size_TR = sum(result["size_TR"] for result in results) / len(results)
    std_size_TR = math.sqrt(sum((result["size_TR"] - mean_size_TR) ** 2 for result in results) / l
    support_fraction = sum(1 for result in results if result["size_TR"] >= result["n"] * (2 ** res

    return {
        "metric_name": "tree_resolution_size",
        "metric_value": mean_size_TR,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction == 1.0,
        "counterexample": "" if support_fraction == 1.0 else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    results = [run_trial(seed) for seed in seeds]
    mean_size_TR = sum(result["metric_value"] for result in results) / len(results)
    std_size_TR = math.sqrt(sum((result["metric_value"] - mean_size_TR) ** 2 for result in results
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction == 1.0:
        print(f"RESULT: SUPPORTED mean={mean_size_TR} std={std_size_TR} support_fraction={support_
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ee706686.py", line 187, in <module>
```

```
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ee706686.py", line 155, in run_trial
    cnf = tseitin_cnf(graph, n, charge)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ee706686.py", line 54, in tseitin_cnf
    clause = [literals[i][charge[i]]]
              ~~~~~~~~~~~^^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with an IndexError in tseitin_cnf before producing any data, so neither the support nor falsification condition can be evaluated. | next: Fix the indexing bug in tseitin_cnf (likely a mismatch between vertex count and the literals/charge arrays) and rerun the 720-instance pre-registered protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	75350
2	preregistration	claude_max	opus	0	0	6617
3	novelty	claude_max	opus	0	0	3920
4	novelty	claude_max	opus	0	0	9111
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27359
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	75384
7	judge	claude_max	opus	0	0	105280

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 303020 ms total latency. Provider mix: {‘claude_max’: 5, ‘ollama_remote’: 2}

(full prompt+response transcripts available in `research/audit/82f0b7df0af9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/82f0b7df0af9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/82f0b7df0af9.tar.gz` (if generated)